AI TEAM · DESIGN BRIEFING

회사 문서를 찾고, 근거로 답하는 AI

AI팀의 기술 설계와 백엔드·프론트 협업 방안,
MVP 구현 일정을 공유합니다.

01 RAG를 쓰는 이유
02 어떻게 동작하나요
03 선택한 모델 2개와 이유
04 백엔드와 조율·데이터·추천안
05 프론트 사용자 경험·조율
06 설계 요약과 구현 계획
07 4주 일정과 MVP 확장 목표



쉽게 말하면 용어 설명 · 1개

온보딩
새로 입사한 사람이 회사의 업무 방식과 시스템을 익히는 과정. 이 AI가 돕고 싶은 목표입니다.

01 · 왜 RAG를 쓰나요

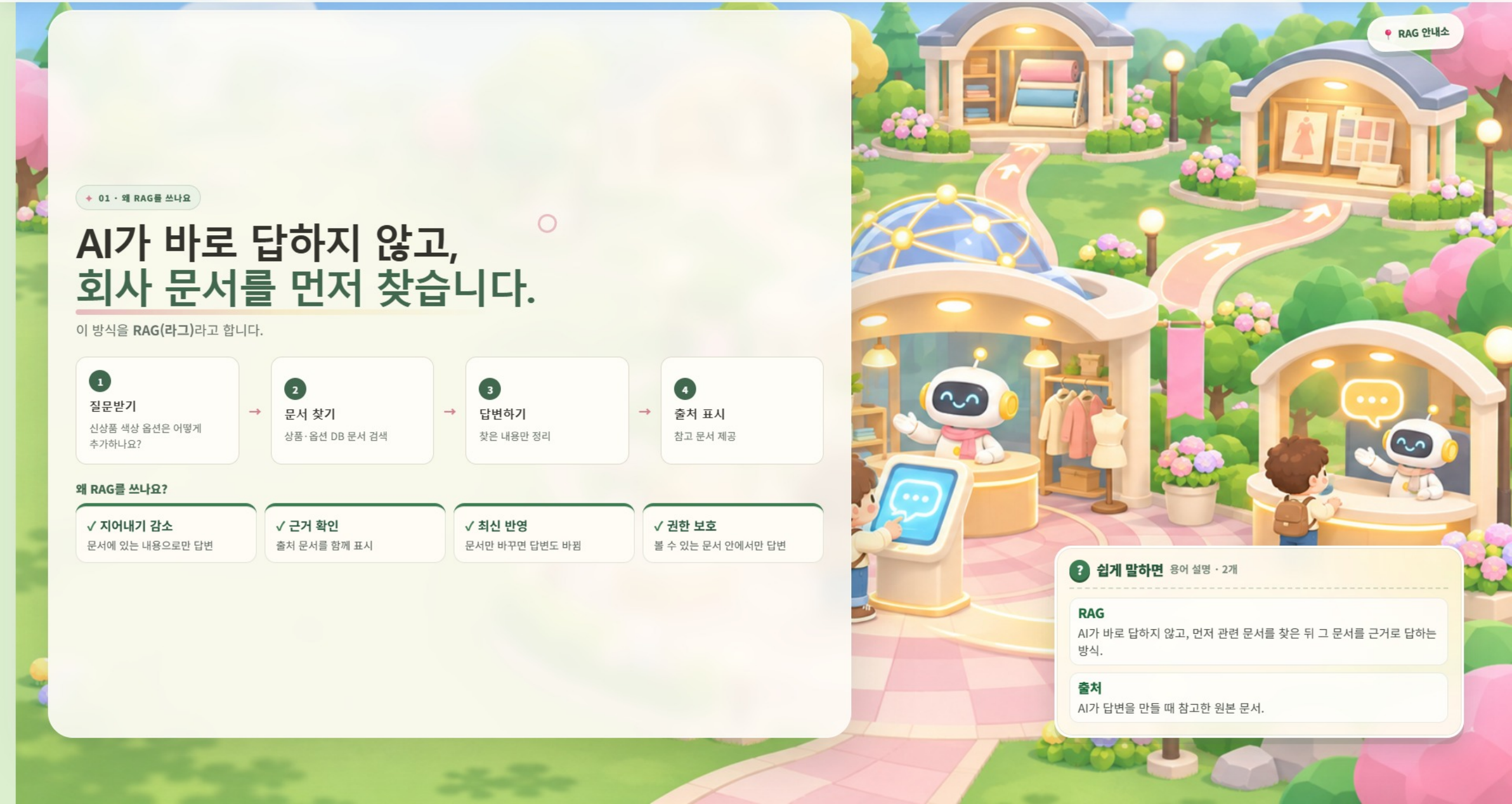
AI가 바로 답하지 않고, 회사 문서를 먼저 찾습니다.

이 방식을 **RAG(라그)**라고 합니다.

1. **질문받기** — 신상품 색상 옵션은 어떻게 추가하나요?
2. **문서 찾기** — 상품·옵션 DB 문서 검색
3. **답변하기** — 찾은 내용만 정리
4. **출처 표시** — 참고 문서 제공

왜 RAG를 쓰나요?

- ✓ **지어내기 감소** — 문서에 있는 내용으로만 답변
- ✓ **근거 확인** — 출처 문서를 함께 표시
- ✓ **최신 반영** — 문서만 바꾸면 답변도 바뀜
- ✓ **권한 보호** — 볼 수 있는 문서 안에서만 답변



? 쉽게 말하면 용어 설명 · 2개

RAG
AI가 바로 답하지 않고, 먼저 관련 문서를 찾은 뒤 그 문서를 근거로 답하는 방식.

출처
AI가 답변을 만들 때 참고한 원본 문서.

02 · 어떻게 동작하나요

문서를 조각으로 나눠 두고, 질문이 오면 관련 조각 5개를 찾습니다.

왜 나누나요?

긴 문서 전체보다 필요한 내용을 빠르고 정확하게 찾기 위해서입니다.

왜 5개인가요?

너무 적으면 내용이 빠지고, 너무 많으면 관계없는 내용이 섞이기 때문입니다.

어떤 과정으로 동작하나요?

1. **미리 나누기**
   문서를 **800자** 조각으로, 앞뒤 **120자** 겹치게
2. **찾기**
   검색 AI가 질문과 가까운 조각 **5개** 선택
3. **답하기**
   답변 AI가 5개 조각만 읽고 정리
4. **출처 붙이기**
   참고한 문서 목록을 함께 전달

※ 800자, 120자, 5개는 시작값이며 테스트 결과에 따라 변경할 수 있습니다.



? 쉽게 말하면 용어 설명 · 4개

청킹
긴 문서를 검색하기 좋은 작은 조각으로 나누는 작업.

문서 조각
긴 문서에서 나눈 작은 내용 단위.

오버랩
문서를 자를 때 앞뒤 내용을 조금 겹치게 해 문맥이 끊어지지 않도록 하는 방법.

TOP 5
질문과 관련성이 높은 문서 조각 5개를 선택한다는 뜻.

02-1 · 문서 예시

예를 들어 개발부 문서라면, 이렇게 찾아서 답합니다.

패션회사 개발부 문서 예시 · 14종

신입 개발자가 AI에 물어볼 수 있는 자료

- 신입 개발자 개발 환경 설정
- 사내 계정과 권한 신청 방법
- Git 브랜치 운영 규칙
- 커밋 메시지 작성 규칙
- 코드 리뷰 기준
- 프론트엔드 코딩 컨벤션
- 백엔드 코딩 컨벤션
- 상품·옵션 DB 명세
- 상품 이미지 업로드 규칙
- 상품 검색 API 문서
- 재고 연동 API 문서
- 주문·결제 장애 대응 방법
- 배포 절차
- 롤백 방법

대표 질문

"신상품에 색상 옵션을 추가하려면 어떤 API와 DB를 수정해야 하나요?"

관련 문서 검색 결과 TOP 5

1. 상품·옵션 DB 명세
2. 상품 등록 API
3. 재고 연동 API
4. 프론트엔드 상품 폼 가이드
5. QA 체크리스트

💡 AI는 전체 문서 중 질문과 관련된 조각 5개를 찾아 Qwen3 8B에 전달하고, Qwen3 8B는 그 내용을 하나의 답변으로 정리합니다.

※ 문서명과 질문은 화면 설명을 위한 예시입니다.



03 · 사용할 모델

핵심은 2가지 모델입니다.

① 문서를 찾는 AI · Hugging Face 공개 모델

bge-m3 BAAI 제작

질문과 의미가 비슷한 회사 문서 조각을 찾습니다.

- 왜? 한국어를 포함한 다국어 검색을 지원
- 왜? 긴 문서도 의미 단위로 비교 가능
- 왜? Hugging Face에서 널리 쓰여 검증된 검색 모델

② 답변을 만드는 AI · Hugging Face 공개 모델

Qwen3 8B Alibaba Qwen팀 제작

찾은 문서 조각을 읽고 자연스러운 한국어 답변을 만듭니다.

- 왜? 한국어 질문 이해와 답변 작성 지원
- 왜? 회사 PC·서버에서 실행 가능한 크기(8B)
- 왜? 무료로 업무에 사용할 수 있는 라이선스

왜 2개가 필요한가요? 문서를 빨리 **찾는 일**과 문장을 자연스럽게 **쓰는 일**은 잘하는 모델이 다르기 때문입니다.

Hugging Face	→	Ollama	→	FastAPI
모델을 받는 곳		모델을 실행		백엔드와 연결

선택한 이유

무료 · 한국어 · 로컬 실행의 균형이 좋기 때문입니다. 단, 실제 속도와 정확도는 테스트 후 확정합니다.



쉽게 말하면 용어 설명 · 7개

bge-m3
질문과 의미가 비슷한 문서를 찾아주는 검색용 AI 모델.

Qwen3 8B
찾은 문서를 읽고 자연스러운 답변을 만드는 생성형 AI 모델.

8B
모델 규모를 나타내는 표현. 약 80억 개의 파라미터를 가진 모델이라는 뜻.

파라미터
AI가 학습하면서 기억한 패턴의 규모를 나타내는 값.

Hugging Face
학습이 완료된 AI 모델을 검색하고 내려받을 수 있는 모델 저장소.

Ollama
다운로드한 AI 모델을 회사 PC나 서버에서 실행하는 프로그램.

FastAPI
Python으로 AI 서버와 API를 만들 때 사용하는 개발 도구.

04 · 백엔드와 조율할 부분

백엔드와는 두 개의 흐름만 맞추면 됩니다.

① 요청 흐름 백엔드 → AI · 질문 + 권한

AI는 검색에만 쓰고 저장하지 않음

사용자 질문 → 백엔드: 로그인·권한 확인 → AI 서버로 전송 → AI: 볼 수 있는 문서만 검색

질문
사용자가 입력한 질문
예: 색상 옵션은 어떻게 추가하나요?

볼 수 있는 문서 범위
이 사람이 봐도 되는 문서
예: 전체 공개, 개발부

이전 대화
앞에서 주고받은 대화 몇 개
예: 직전 질문과 답변

② 응답 흐름 AI → 백엔드 → 화면 · 답변 + 출처 (실시간 전송)

AI: 답변 생성 → 답변 조각 계속 전송 → 완료 신호 (완성 답변·참고 문서) → 백엔드: 화면 전달 · DB 저장

답변 조각
답변을 잘라서 계속 보낸 것
화면 ✓ 바로 표시
DB ✗ 저장 안 함

참고 문서 목록
문서 제목 · 관련 문장 · 원문 위치
화면 ✓ 출처 카드
DB ✓ 답변과 함께

완료 신호
끝! + 완성된 답변 전체
화면 ✓ 작성 중 표시 끝
DB ✓ 완성 답변 저장

오류 신호
실패 + 이유
화면 ✓ 안내 문구
DB 선택 오류 기록

※ 조각으로 나누는 건 **화면에 실시간으로 보여주기 위해서**입니다. 답변이 몇 초 안에 나오면 완성된 답변을 한 번에 보내도 됩니다 → 1주차 속도 테스트 후 결정

핵심 요청 **3가지**와 응답 **4가지 신호**의 이름과 형식만 약속하면, 백엔드와 AI는 **각자 따로 개발**하고 마지막에 바로 연결할 수 있습니다.

협업 광장

? 쉽게 말하면 용어 설명 · 8개

파이프라인
데이터가 정해진 순서대로 거쳐 가는 흐름. 여기서는 요청 흐름과 응답 흐름 2개입니다.

실시간 전송(SSE)
연결을 열어 두고 답변이 만들어지는 대로 조금씩 계속 보내는 방식.

답변 조각
답변을 잘라서 보낸 것. 조각을 이어 붙이면 전체 답변이 됩니다.

참고 문서 목록
AI가 답변할 때 본 문서 이름들. 화면의 출처 카드가 됩니다.

완료 신호
답변을 다 보냈다는 신호. 이때 완성된 답변을 DB에 저장합니다.

오류 신호
처리 중 문제가 생겼다는 신호. 화면에 안내 문구를 띄웁니다.

볼 수 있는 문서 범위
이 사용자가 봐도 되는 문서의 범위. AI는 이 안에서만 찾습니다.

이전 대화
앞에서 사용자와 AI가 주고받은 대화. 이어지는 질문을 이해할 때 씁니다.

04-1 · 백엔드와 AI가 다루는 데이터

저장소는 따로 쓰고, 주고받는 것만 약속합니다.

백엔드가 관리

백엔드 DB · 대화 기록장

- **사용자** — 이름, 부서
- **권한** — 볼 수 있는 문서 범위
- **대화방** — 누가, 언제 시작했는지
- **질문과 완성된 답변** — 내용, 시간
- **참고 문서 목록** — 답변마다 본 문서
- (선택) **피드백·오류 기록** — 👍/👎, 실패 기록

주고받기만 · 저장 안 함

백엔드 → AI

질문 · 볼 수 있는 문서 범위 · 이전 대화

AI → 백엔드

답변 조각 · 참고 문서 목록 · 완료 신호 · 오류 신호

AI팀이 관리

AI 저장소 · 문서 서랍

- **문서 조각** — 회사 문서를 800자씩 자른 글
- **검색용 데이터** — 조각의 뜻을 숫자로 바꾼 것
- **조각 정보** — 어느 문서에서 왔는지, 수정일
- **공개 범위** — 전체 공개, 개발부 등
- (선택) **평가 기록** — 찾은 조각, 답변 시간

같은 PostgreSQL을 쓴다면

- **pgvector** 기능을 켜서 AI 검색도 가능
- DB 안을 **칸(스키마) 2개**로 나눔: backend / ai
- **계정도 따로**: 각자 자기 칸만 사용

누가 관리하나요?

- **DB 자체**(설치·백업·계정) → 백엔드
- **AI 칸 안의 내용**(문서 조각·인덱싱) → AI팀
- 꼭 맞출 것: **공개 범위 이름** (예: 개발부 = dev)

협업 광장 · 자료 보관소



쉽게 말하면 용어 설명 · 6개

벡터 저장소
문서 조각을 뜻이 비슷한 것끼리 빨리 찾을 수 있게 저장하는 곳. AI의 문서 서랍입니다.

검색용 데이터
문장의 뜻을 숫자로 바꾼 것(임베딩). 질문과 비슷한 조각을 찾을 때 씁니다.

PostgreSQL
많이 쓰이는 데이터베이스 프로그램. 백엔드 DB 후보입니다.

pgvector
PostgreSQL에서 뜻으로 검색할 수 있게 해 주는 확장 기능.

스키마
같은 DB 안을 나누는 칸. backend 칸, ai 칸처럼 나눠 씁니다.

공개 범위
문서마다 붙이는 표시(전체 공개, 개발부 등). 백엔드와 이름을 똑같이 맞춰야 합니다.

04-2 · 백엔드와 조율할 부분

백엔드와 정해야 할 6가지, AI팀 제안과 백엔드가 할 일입니다.

	무엇을 정하나요?	AI팀 제안	백엔드가 할 일
1	**질문 보내기** API 명세 · 요청 백엔드가 AI에 질문을 어떻게 보낼까?	AI 서버 주소 `/ai/chat` 하나로 **3가지**를 보냄: ① 질문 ② 볼 수 있는 문서 범위 ③ 이전 대화(최근 6개)	**사용자가 질문하면 이 3가지를 담아 AI에 요청**
2	**답변 받기** API 명세 · 응답 AI는 답변을 어떤 순서로 돌려줄까?	① **답변 조각**을 여러 번 → ② 끝나면 **완료 신호**(완성 답변 + 참고 문서) · 실패하면 **오류 신호**	**조각은 화면에 바로 넘기고, 완료 신호를 받으면 DB에 저장**
3	**권한** 명세에 포함 어떤 문서까지 찾아도 되는지 어떻게 알려줄까?	로그인한 사람이 볼 수 있는 범위를 **이름**으로 보냄 (예: 전체 공개 = public, 개발부 = dev). AI는 그 문서만 검색	**사용자별 범위 이름을 정하고, 질문마다 함께 보내기**
4	**저장소** 운영 약속 데이터를 어디에 저장할까?	같은 PostgreSQL 안에 **칸 2개**(백엔드 칸 / AI 칸), 계정도 따로 → 서로의 데이터는 건드리지 않음	**DB 설치·백업 관리, AI용 칸과 계정 만들어 주기**
5	**문서 갱신** 운영 약속 AI가 읽을 회사 문서는 어디서 가져오고, 바뀌면 어떻게 할까?	문서를 **한 폴더**에 모아 두면 AI가 **매일 밤 1번** 다시 읽어 정리 → **오늘 고친 문서는 내일부터 답변에 반영** (급하면 바로 다시 읽기)	**문서 폴더 위치를 정해 공유하기**
6	**느리거나 실패할 때** 명세에 포함 AI가 너무 느리거나 실패하면 어떻게 할까?	**10초** 안에 첫 글자가 없거나 **60초** 안에 끝나지 않으면 실패로 처리. 실패 이유 4가지: 시간 초과 · 문서 없음 · 모델 오류 · 잘못된 요청	**실패 이유에 맞는 안내 문구를 화면에 전달**

정하는 방법 명세 항목은 API 명세 문서, 운영 약속은 운영 문서로 정리 → 샘플 질문 3개로 **함께 테스트** → **1주차 안에 확정** · ※ 숫자는 제안



쉽게 말하면 용어 설명 · 7개

엔드포인트
백엔드가 AI 서버에 질문을 보내는 주소. 예: /ai/chat

JSON
프로그램끼리 질문, 권한, 답변 등의 데이터를 주고받는 형식.

실시간 전송 신호
답변을 조금씩 보낼 때 붙이는 이름표. 답변 조각·완료·오류로 나눠 보냅니다.

권한 범위
사용자가 볼 수 있는 문서의 범위. 백엔드가 질문과 함께 AI에 전달합니다.

재인덱싱
문서가 바뀌었을 때 검색용 데이터를 다시 만드는 작업.

타임아웃
정해진 시간 안에 응답이 없으면 실패로 처리하는 기준.

오류 코드
어떤 문제가 생겼는지 프로그램끼리 알아볼 수 있게 붙이는 약속된 이름.

05 · 프론트 사용자 경험

기술 용어보다,
지금 무엇을 하는지 보여줍니다.

1. **질문 표시** 프론트 직접
   사용자가 보낸 질문을 바로 보여줌
2. **기다리는 동안** 프론트 직접
   “답변을 준비하고 있어요” 안내
3. **실시간 답변** 답변 조각
   오는 대로 글자를 이어 붙여 표시
4. **출처 표시** 참고 문서 목록
   문서 제목·관련 문장·원문 열기
5. **실패 안내** 오류 신호
   이유별 안내 + 다시 시도 버튼

화면 예시 화면 동작을 설명하기 위한 예시

궁금한 개발 문서 내용을 물어보세요.

신상품에 새로운 색상 옵션을 추가하려면 어떻게 해야 하나요?

먼저 `product_options` 테이블에 색상 옵션을 추가하고, 상품 등록 API의 `options` 필드에 해당 값을 포함해야 합니다. 이후 상품 등록 화면과 재고 연동 여부를 함께 확인해야 합니다.

참고 문서 5개 · 예시
- 상품·옵션 DB 명세
- 상품 등록 API 문서
- 재고 연동 API 문서
- 프론트엔드 상품 폼 가이드
- QA 체크리스트

※ 화면 동작을 설명하기 위한 예시이며, 실제 확정된 회사 규칙이 아닙니다.

프론트의 핵심

AI가 보내는 신호는 **답변 조각 · 완료 · 참고 문서 · 오류**뿐이고, 모두 **백엔드를 거쳐** 화면에 옵니다.



쉽게 말하면 용어 설명 · 3개

Streaming
완성된 답변을 한 번에 보여주지 않고 생성되는 순서대로 조금씩 보여주는 방식.

출처 카드
AI가 참고한 문서의 제목과 내용을 확인할 수 있는 화면 요소.

재시도
오류가 발생했을 때 같은 요청을 다시 보내는 기능.

05-1 · 프론트와 조율할 부분

프론트와 정해야 할 6가지, AI팀 제안과 프론트가 할 일입니다.

무엇을 정하나요?	AI팀 제안	프론트가 할 일
1 **답변 표시** 답변 조각 답변을 어떻게 보여줄까?	답변 조각이 올 때마다 **바로 이어 붙이고,** 쓰는 중에는 커서 깜빡임	**조각을 받는 즉시 화면에 추가**
2 **기다리는 동안** 프론트 직접 첫 글자가 오기 전에는 뭘 보여줄까?	**"답변을 준비하고 있어요"** 문구 하나 (단계별 문구가 필요하면 진행 상태 신호를 추가로 협의)	**질문을 보내자마자 문구 표시**
3 **출처 카드** 참고 문서 목록 참고한 문서를 어떻게 보여줄까?	답변 아래에 **문서 제목 + 관련 문장 1~2줄 + 원문 열기**	**완료 신호를 받으면 카드 표시**
4 **실패 화면** 오류 신호 실패하면 뭘 보여줄까?	실패 이유 4가지별 **안내 문구 + 다시 시도 버튼** + 담당자 문의	**오류 신호를 받으면 안내 표시**
5 **대화 기록** 백엔드 DB 지난 대화는 어떻게 보여줄까?	왼쪽에 **최근 대화 목록**과 새 대화 버튼	**백엔드에서 목록을 받아 표시**
6 **답변 평가** 백엔드 DB 답변이 맞았는지 어떻게 모을까?	답변마다 👍 / 👎, 👎이면 이유 선택 (틀림 · 부족 · 출처 없음)	**누르면 백엔드에 저장 요청**

정하는 방법 화면 시안 + 샘플 응답으로 **목업**을 만들어 함께 확인 → **1주차 안에 확정** · ※ 문구와 숫자는 제안



? 쉽게 말하면 용어 설명 · 6개

답변 조각
답변을 잘라서 보낸 것. 오는 대로 이어 붙이면 실시간으로 써지는 것처럼 보입니다.

완료 신호
답변을 다 보냈다는 신호. 이때 출처 카드를 띄웁니다.

참고 문서 목록
AI가 답변할 때 본 문서들. 제목, 관련 문장, 원문 위치를 담습니다.

오류 신호
처리 중 문제가 생겼다는 신호. 실패 이유가 함께 옵니다.

목업
실제 서버 없이 가짜 데이터로 화면 동작을 미리 확인하는 방식.

답변 평가
사용자가 답변이 도움이 됐는지 👍/👎로 표시하는 기능. 정확도 개선에 씁니다.

06 · 지금까지의 설계와 앞으로의 구현

설계는 끝났고,
결정만 되면 바로 구현합니다.

✅ 지금까지 설계한 것

- ✓ RAG 방식으로 근거 답변
- ✓ bge-m3 검색 + Qwen3 8B 답변
- ✓ 800자 청킹 · TOP 5 검색
- ✓ 백엔드 ↔ AI 역할 분리
- ✓ 실시간 답변 + 출처 표시

→

🤝 함께 결정할 것

백엔드	API 명세 v1 · 실시간 전송 신호
백엔드	공개 범위 이름 · DB 관리 분담
백엔드	문서 폴더 위치
프론트	화면 시안 · 로딩 문구
프론트	출처 카드 · 오류 화면

→

🛠 이렇게 구현합니다

1. **문서 인덱싱**
   문서 불러오기 → 800자 청킹 → bge-m3 임베딩 → 벡터 저장소
2. **검색·답변 API**
   FastAPI + Ollama(Qwen3 8B), SSE로 전송
3. **연동**
   백엔드 API 연결, 프론트 스트리밍 화면
4. **평가**
   실제 질문 50개로 정확도·속도 측정

> “모델이 좋다고 가정하지 않고,
> 실제 회사 문서로 속도와 정확도를 확인한 뒤 확정하겠습니다.”

※ 벡터 저장소는 후보(Chroma, FAISS 등)를 테스트한 뒤 결정합니다.



? 쉽게 말하면 용어 설명 · 9개

RAG
관련 문서를 찾은 후 그 문서를 근거로 답변하는 전체 방식.

bge-m3
질문과 관련된 문서를 찾는 검색 모델.

Qwen3 8B
검색된 문서를 읽고 답변을 작성하는 생성 모델.

인덱싱
문서를 검색할 수 있도록 미리 나누고 변환해 저장하는 작업.

임베딩
문장의 의미를 숫자 목록으로 바꾸는 것. 의미가 비슷한 문서를 찾을 때 씁니다.

벡터 저장소
임베딩한 문서 조각을 저장하고 빠르게 찾아주는 저장소.

SSE
AI가 생성 중인 답변을 실시간으로 프론트에 전달하는 방식.

정확도 평가
AI가 질문에 맞는 문서를 찾고 올바른 답변을 만드는지 실제 질문으로 확인하는 과정.

성능 테스트
AI 답변 속도, 서버 자원 사용량과 동시 처리 가능 인원을 확인하는 테스트.

07 · 4주 일정과 다음 목표

4주 안에 설계를 마무리하고 MVP를 구현합니다.

1주 · 설계 마무리	2~3주 · 구현	4주 · MVP 완성
명세 확정 · 모델 테스트	**검색 · 답변 · 연동**	**질문 → 근거 답변 → 출처**
백엔드·프론트와 약속 확정	문서 인덱싱, 실시간 전송, 화면 연결	개발부 문서로 처음부터 끝까지 동작

작업	1주	2주	3주	4주
설계 마무리·명세 확정 백엔드·프론트	1주			◆ MVP 완성
모델 속도·한국어 테스트 AI	1주			
문서 인덱싱·검색 구현 AI	2주			
답변 생성·실시간 전송 AI		2주		
백엔드 연동 AI + 백엔드			1주	
프론트 화면 연동 AI + 프론트			2주	
통합 테스트·정확도 점검 전체				1주

MVP 이후 목표 · 확장: 권한별 검색 강화 · 대화 기록·답변 평가 · 문서 자동 갱신 · 다른 부서 문서 · 속도·동시 사용자 개선

※ 예상 일정이며 협의 결과와 모델 테스트 결과에 따라 조정합니다.



쉽게 말하면 용어 설명 · 6개

MVP
꼭 필요한 기능만 먼저 만들어 실제로 써 보는 첫 번째 버전.

간트차트
작업별로 언제 시작하고 끝나는지 막대로 보여주는 일정표.

마일스톤
일정 중 꼭 도달해야 하는 지점. 여기서는 4주 차 MVP 완성입니다.

연동
따로 만든 AI 서버, 백엔드, 프론트를 연결해 함께 동작하게 하는 작업.

통합 테스트
모두 연결한 상태에서 질문부터 답변까지 처음부터 끝까지 확인하는 테스트.

확장
MVP를 써 보며 나온 의견으로 기능과 대상 문서를 넓혀 가는 단계.